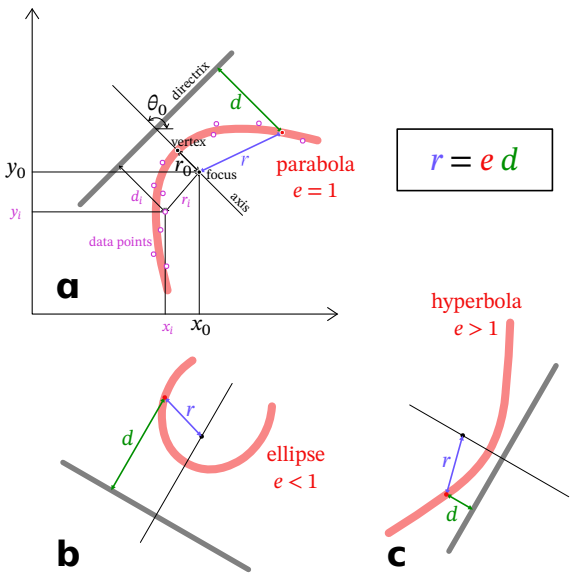

Fig. B.1. Illustration of the procedure for fitting conic sections (thick red curves) to a set of observed points (small open circles) by making use of the focus-directrix relation: $r = ed$. (a) Case of parabola ($e = 1$), labelled with all the relevant quantities as described in the text. (b) Case of ellipse ($e < 1$). Note that only a portion of the ellipse is shown. (c) Case of hyperbola ($e > 1$).

Appendix B: Algorithm for fitting a parabola or other conic section to a set of points

We here present an algorithm for fitting a conic section curve (ellipse, parabola, hyperbola) to a set of points (x_i, y_i), $i \in \{1, \ldots, N\}$, which lie in a two-dimensional plane. This is implemented in the `confitti` python library, which is freely available from the Python Package Index[10] or from GitHub[11]. This description corresponds to version 0.2.1 of the library. The algorithm is a spiritual successor to the `circle-fit` method described in Appendix E of Tarango-Yong & Henney (2018).

Appendix B.1: Conic section parameters

A conic section curve can be completely specified by a set of 5 parameters: $\{x_0, y_0, r_0, \theta_0, e\}$, where (x_0, y_0) is the position of the focus, r_0 is the size, θ_0 is the orientation angle of the curve's symmetry axis, and e is the eccentricity of the conic section. Hyperbolae have $e > 1$, a parabola has $e = 1$, while ellipses have $e < 1$, with $e = 0$ corresponding to a circle (in which case θ_0 is undefined). The size r_0 is taken to be the distance between the focus and the apex (see Figure B.1), which is the point on the curve that is closest to the focus. The symmetry axis passes through both the focus and the apex, with θ_0 being measured anticlockwise from the x axis.

Appendix B.2: Minimization of focus-directrix residuals

The algorithm makes use of the focus-directrix property of conic section curves (e.g., Brannan et al. 2012 § 1.1.3):

$$r = ed, \tag{B.1}$$

where r is the distance from the focus to a point P on the curve, and d is the perpendicular distance of the point P from the directrix. The directrix is a line perpendicular to symmetry axis that crosses the axis at a distance $(1 + e^{-1})\, r_0$ from the focus (see Figure B.1). For a general point (x_i, y_i), which does not necessarily lie on the curve, we can calculate a residual difference $\delta_i = r_i - ed_i$, where

$$r_i = \left((x_i - x_0)^2 + (y_i - y_0)^2\right)^{1/2} \tag{B.2}$$

and

$$d_i = (1 + e^{-1})\, r_0 - (x_i - x_0)\cos\theta_0 - (y_i - y_0)\sin\theta_0. \tag{B.3}$$

We can then define the best-fitting conic section curve as the set of parameters $\{x_0, y_0, r_0, \theta_0, e\}$ that minimizes the sum of squared, scaled residuals:

$$S = \sum_{i=1}^{N} (\delta_i/\sigma_i)^2, \tag{B.4}$$

where the scaling is by the uncertainties σ_i in the (x_i, y_i) measurements (if available).

Appendix B.3: Implementation details

The `confitti` library provides the function `confitti.fit_conic_to_xy(xpts,ypts)` with arguments `xpts` and `ypts`, which correspond to the observed data points (x_i, y_i) discussed above. This function takes care of making an initial guess for the conic section curve parameters, and then refining this guess by minimizing equation (B.4) using the Levenberg-Marquardt algorithm (Moré 1978) as implemented by the `scipy` (Virtanen et al. 2020) and `lmfit` (Newville et al. 2024) Python libraries. The initial guess for the parameters is as follows:

1. The focus point (x_0, y_0) is initialized to the mean position of the data points.
2. The size r_0 is initialized to the mean distance of the 5 closest data points from the initial focus.
3. The orientation θ_0 is initialized to the circular mean angle of the 5 closest data points from the initial focus.
4. The eccentricity is initialized to $e = 1$.

Extensive testing shows that this method works well, so long as the observed data points cover both sides of the apex. Typically, 20 to 50 evaluations of the residual function are required to converge the solution to the default relative tolerance of $\sim 10^{-8}$.

Optionally, the fit may be constrained in various ways. For example, the `only_parabola` option freezes the eccentricity at $e = 1$, such that the fitted curves have only 4 free parameters instead of 5. Explicit per-point uncertainties in the data points, σ_i, may also be provided, but by default these are set to unity. Additionally the library provides functions for saving and loading the fit results in JSON and YAML formats, and for plotting the fitted curves. All features are demonstrated in example jupyter notebooks.[12]

[10] `https://pypi.org/project/confitti` or `pip install confitti`
[11] `https://github.com/div-B-equals-0/confitti`
[12] `https://github.com/div-B-equals-0/confitti/tree/main/notebooks`

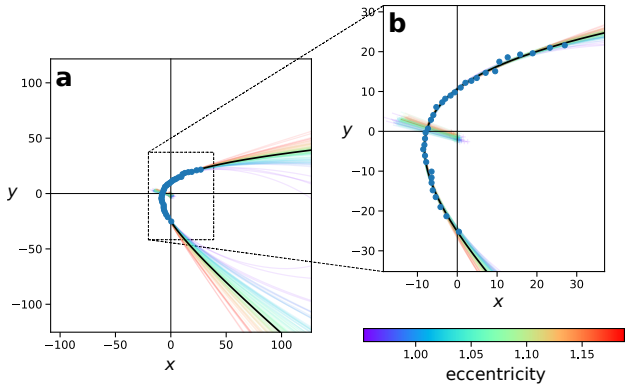

Fig. B.2. Example fit of conic section arc to data points (blue symbols). (a) Zoomed out view. (b) Zoomed in view. Black line shows best-fit curve. Colored lines show a random sample of curves from the MCMC posterior distribution of the conic parameters, colored according to the eccentricity (ellipses in purple, parabola in blue, hyperbolae in green-yellow-red). Also shown are the focus point of each curve (plus symbols) and a straight line that indicates the portion of the symmetry axis between the focus and the directrix, both colored in the same way.

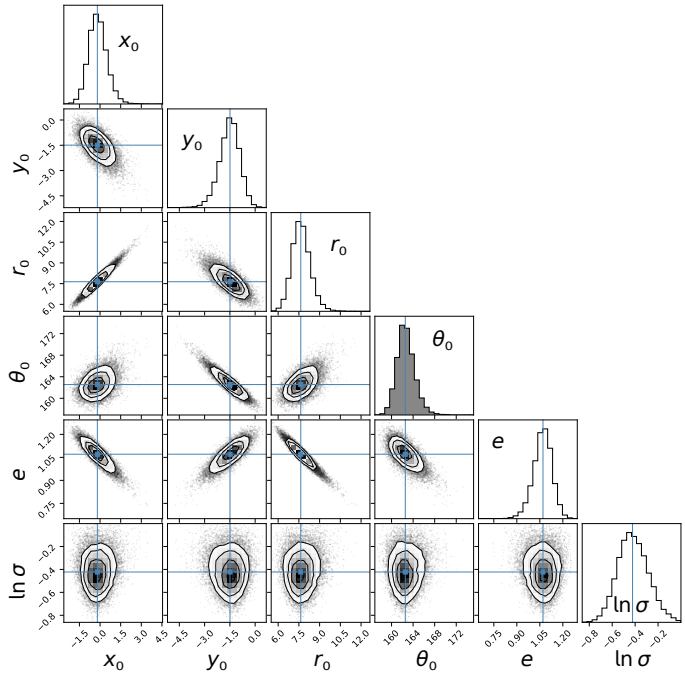

Fig. B.3. Corner plot of MCMC-derived covariances between fitted model parameters from the same example as shown in Figure B.2. Plots on the leading diagonal show the one-dimensional histogram of the posterior distribution of each parameter, assuming a uniform prior distribution. Off-diagonal plots show the two-dimensional histogram of the joint posterior distribution of each pair of parameters. Blue dots and lines indicate the best-fit parameters. The bottom row shows the natural logarithm of the estimated magnitude of observational errors for the individual data points (assumed constant for all points), which is included as an additional parameter in the model.

Appendix B.4: Quantifying uncertainty in derived parameters

In addition to the estimated best-fit parameters, the Levenberg-Marquardt minimization provides a χ^2 measure of the goodness of fit[13] together with a covariance matrix that expresses the estimated uncertainties in the fitted parameters and the correlations between these. Alternatively, the posterior probability distribution of the fitted curve parameters can be explored more thoroughly through Markov Chain Monte Carlo (MCMC) ensemble sampling (Goodman & Weare 2010) as implemented via `lmfit`'s interface to the `emcee` Python library (Foreman-Mackey et al. 2013). By default, we assume uniform priors for all the parameters. In the case that the data-point observational errors σ_i are not known a priori, then a uniform error $\ln\sigma$ can be added as an additional parameter to the model, allowing the posterior probability distribution of the data errors to be estimated also. Note that the MCMC calculation is relatively computationally expensive, requiring of order 5×10^5 function evaluations, which is an increase in effort of 4 orders of magnitude, compared with the Levenberg-Marquardt minimization.

An example is shown in Figure B.2, where colored lines show curves with parameters taken from a random selection of 100 posterior samples from the MCMC chain, which gives an estimate of the uncertainty around the best-fit curve (black line). Figure B.3 shows the pairwise correlations and marginal histograms of the posterior distributions of conic parameters from the same example fit, expressed as a corner plot (Foreman-Mackey 2016).

Appendix B.5: Comparison with previous work

A variety of different approaches to conic fitting are described in Zhang (1997), with the two main categories being algebraic and geometric methods. In the algebraic approach, the conic curve is a solution of the quadratic equation:

$$Ax^2 + 2Bxy + Cy^2 + 2Dx + 2Ey + F = 0, \tag{B.5}$$

[13] The absolute value of χ^2 is only meaningful in the case that explicit uncertainties σ_i in the data points are provided.

together with a normalization constraint. For instance, with the normalization $F = 1$, then the curve is specified by the parameters $\{A, B, C, D, E\}$. One disadvantage of algebraic methods is that data points in high-curvature sections of the conic have less influence on the solution than points in low-curvature sections. A second disadvantage is that the parameters $\{A, B, C, D, E\}$ transform in a complicated way under translations and rotations of the axes, unlike our preferred parameters $\{x_0, y_0, r_0, \theta_0, e\}$ (section B.1), which have a more intuitive interpretation in terms of the conic shape.

The most obvious geometric method is to minimize the sum of squared orthogonal distances of the points from the curve (Zhang 1997, § 5.2). However, computation of the orthogonal distance is complicated and requires solving a quartic equation. We therefore prefer to instead make use of the much simpler focus-directrix relation described above (section B.2). Our method is closely related to the geometric fitting technique proposed by López-Rubio et al. (2018) (LR18), but with the following key differences.

1. Our method applies to general conics, while LR18 is restricted to parabolas.
2. LR18 employed a bespoke minimization algorithm, with explicit analytic calculation of the gradients of the errors, and including multiple restarts of the process from a random perturbation of an initial solution. In our case we prefer to employ a general-purpose off-the-shelf minimization library with numerical estimates of the gradients, and replacing the random restarts with the option to perform MCMC ensemble sampling.

3. LR18 added an additional penalty term to the cost function in order to avoid degenerate solutions, whereas we find that the same goal can be achieved by constraining the conic size to lie within a factor of 3 of the initial guess.
4. LR18 use minimization of absolute errors to improve the robustness of the method, whereas we find that regular minimization of mean-square errors is sufficient.

Appendix C: The outflows

Guzmán Ccolque et al. (2024) used ALMA observation and reported 18 CO outflow in DR 21, all seen in Fig. C.1. They show that 10 are blue-shifted and 8 are red-shifted, hence the color code in Fig. C.1. All the outflows seem to project in the direction of the center of DR 21 core.

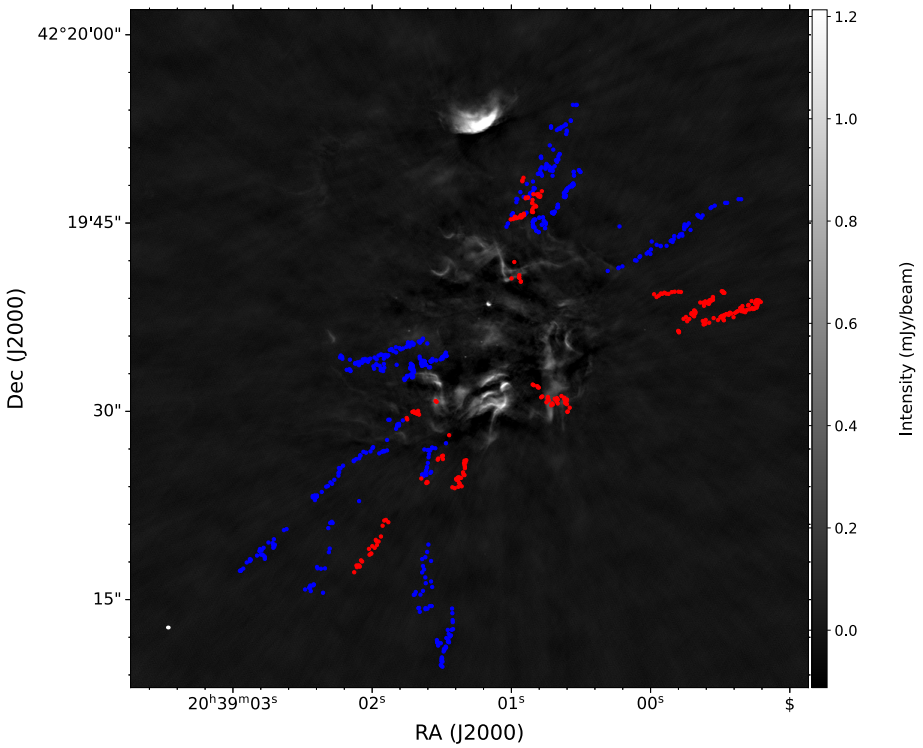

Fig. C.1. VLA radio image towards DR21 in gray scale. Dots show the blue and redshifted streamers detected with ALMA by Guzmán Ccolque et al. (2024), and colored accordingly.